

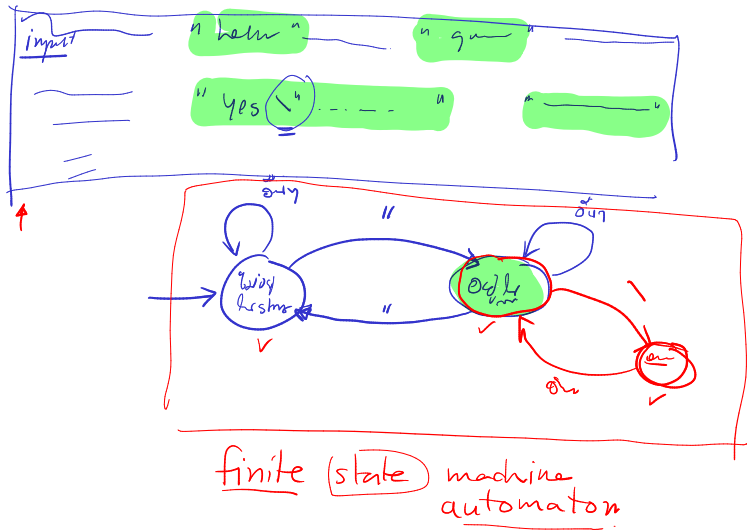
01204211 Discrete Mathematics
Lecture 9a: Finite automata¹

Jittat Fakcharoenphol

August 25, 2026

¹Based on lecture notes of *Models of Computation* course by Jeff Erickson. 

Example: syntax highlighting

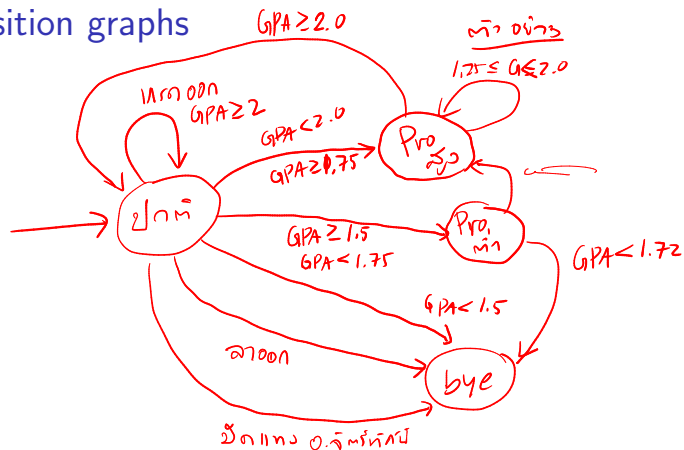


HTML tokenizer

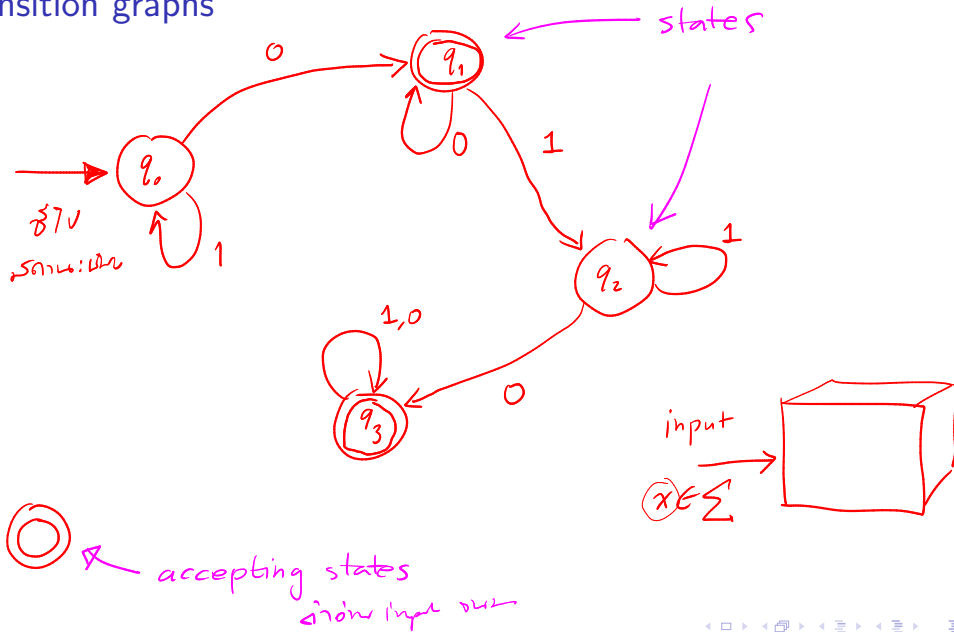
Game programming

design pattern → State

State-transition graphs

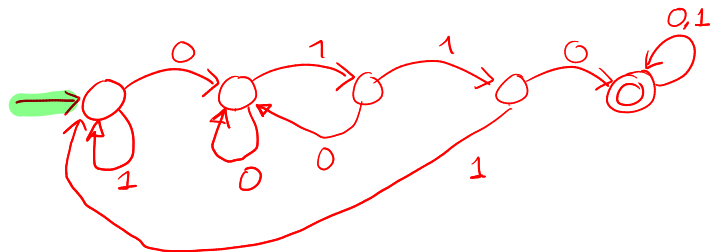


State-transition graphs



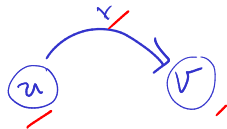
State-transition graphs

(c) strings that contain substring 0110

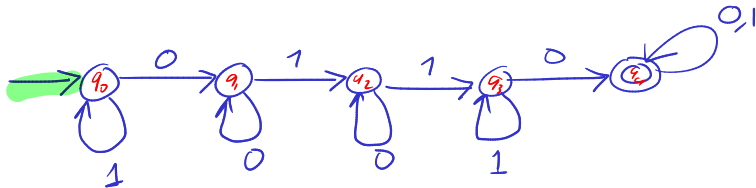


Regex

$(0+1)^*0110(0+1)^*$



(d) $stmn$ ————— Subsequence 0110

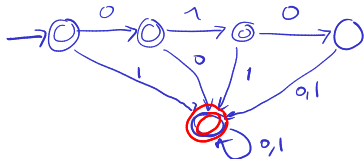


$$\delta(q_3, 0) = q_4$$

$\uparrow$ $\uparrow$
san in

More examples over $\Sigma = \{0, 1\}$

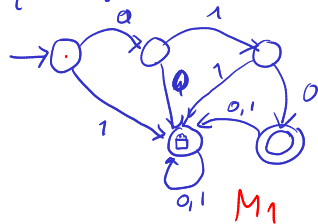
All strings, except 010.



Strings containing the subsequence 010.

All strings $\rightarrow$ A simple DFA with one state that is both the start and the only accepting state, with a self-loop for both 0 and 1.

$\{010\}$



M_1

01 No

010 Yes

0101 No

Formal definitions

A **finite-state machine** or a **deterministic finite-state automaton** (DFA) has five components:

Formal definitions

A **finite-state machine** or a **deterministic finite-state automaton** (DFA) has five components:

- ▶ the input alphabet Σ ,

Formal definitions

A **finite-state machine** or a **deterministic finite-state automaton** (DFA) has five components:

- ▶ the input alphabet Σ ,
- ▶ a finite set of states Q ,

Formal definitions

A **finite-state machine** or a **deterministic finite-state automaton** (DFA) has five components:

- ▶ the input alphabet Σ ,
- ▶ a finite set of states Q ,
- ▶ a transition function δ

Formal definitions

A **finite-state machine** or a **deterministic finite-state automaton** (DFA) has five components:

- ▶ the input alphabet Σ ,
- ▶ a finite set of states Q ,
- ▶ a transition function $\delta : Q \times \Sigma \longrightarrow Q$

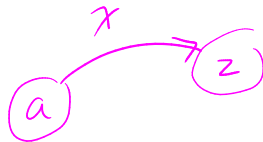
Formal definitions

$$Q \times \Sigma$$

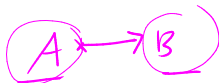
no dom input

A **finite-state machine** or a **deterministic finite-state automaton** (DFA) has five components:

- ▶ the input alphabet Σ ,
- ▶ a finite set of states Q ,
- ▶ a transition function $\delta: \underbrace{Q \times \Sigma}_{\text{domain}} \rightarrow \underbrace{Q}_{\text{range}}$
- ▶ a start state $s \in Q$, and
- ▶ a subset $A \subseteq Q$ of accepting states.



$$\delta(a, x) = z$$



Formal definitions

A **finite-state machine** or a **deterministic finite-state automaton** (DFA) has five components:

- ▶ the input alphabet Σ ,
- ▶ a finite set of states Q ,
- * ▶ a transition function $\delta: Q \times \Sigma \rightarrow Q$
- ▶ a start state $s \in Q$, and
- ▶ a subset $A \subseteq Q$ of accepting states.

input string w

DFA accepts if and only if

final state $(s, w) \in A$

↑ initial ↑ input



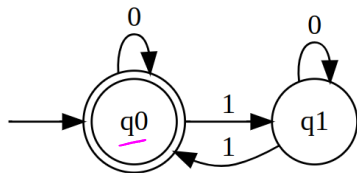
if $w = \epsilon$

final state $(s, \epsilon) = s$

if $w = x^1 \dots x^n$ $x \in \Sigma$

final state $(s, x) = \delta(s, x)$

Example 1



$(\underline{\Sigma}, \underline{Q}, \underline{\delta}, \underline{q_0}, \{\underline{q_0}\})$
Darin' $\Sigma = \{0, 1\}$

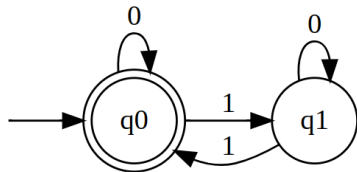
$Q = \{q_0, q_1\}$

δ	0	1
q_0	q_0	q_1
q_1	q_1	q_0

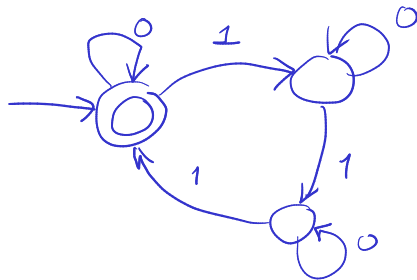
684 $\delta(q_0, 0) = q_0$

$\delta(q_1, 0) = q_1$

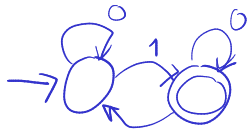
Example 1



DFA អ្វីហ្នឹង string ពុំឆ្គង 2
ឬឆ្គង 3 ឆ្គង.

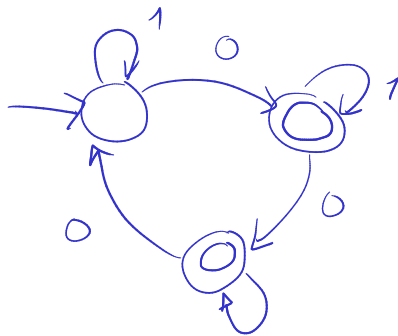
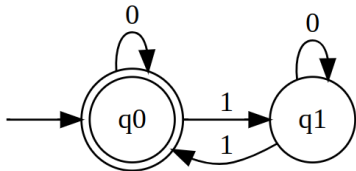


Example 1

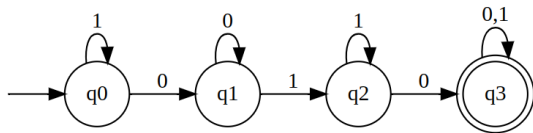


M_1 ทุก string ที่ จน. 1 เป็นดี

M_2 ทุก string ที่ จน. 0 นบด้วย 3 7 ไม่ลงตัว.



Example 2



$(\{0, 1\},$
 $\{q_0, q_1, q_2, q_3\},$
 $\delta,$
 $q_0,$
 $\{q_3\})$

តារាង

δ អនុវត្តការប្រែប្រួល

	0	1
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_3	q_3

Moves

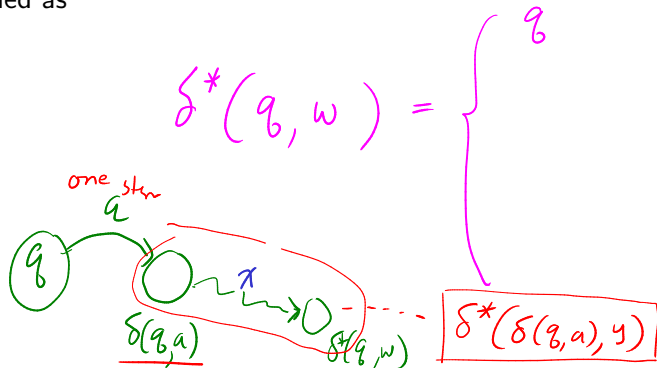
One step move: from state q with input symbol a , the machine changes its state to

Moves

string w $\begin{cases} \epsilon \\ a.y \text{ iff } a \in \Sigma, \\ y \text{ is string.} \end{cases}$

One step move: from state q with input symbol a , the machine changes its state to $\delta(q, a)$.

Extension: from state q with input string w , the machine changes its state to $\delta^*(q, w)$ defined as



$w = \epsilon$

$w = \underline{a}.x$
 $a \in \Sigma$
 $x \in \Sigma^*$

Moves

One step move: from state q with input symbol a , the machine changes its state to $\delta(q, a)$.

Extension: from state q with input string w , the machine changes its state to $\delta^*(q, w)$ defined as

$$\delta^*(q, w) = \begin{cases} q & \text{if } w = \varepsilon, \\ \delta^*(\delta(q, a), x) & \text{if } w = ax. \end{cases}$$

The signature of δ^* is $Q \times \Sigma^* \rightarrow Q$.



Acceptance

For a finite-state machine with starting state s and accepting states A , it accepts string w iff

$$\delta^*(s, w) \in A$$

Acceptance

For a finite-state machine with starting state s and accepting states A , it accepts string w iff

$$\delta^*(s, w) \in A.$$

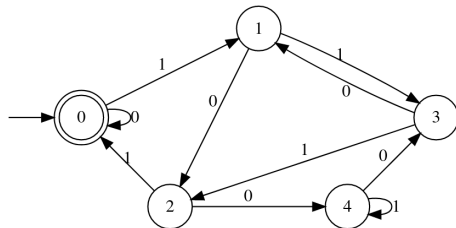
Multiple of 5

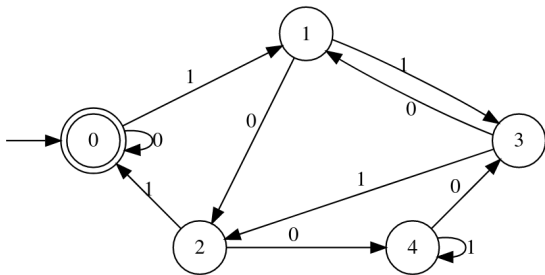
Multiple of 5

```
def multiple_of_5(w):  
    r = 0  
    for i in w:  
        r = (2*r + w) % 5  
    return r == 0
```

Multiple of 5

```
def multiple_of_5(w):  
    r = 0  
    for i in w:  
        r = (2*r + w) % 5  
    return r == 0
```





Digital design: Implementation

Digital design: Moore and Mealy machines

In the digital design class, you will encounter finite-state machines as well. The version we consider in this class is referred to as a **Moore machine**.

In practice, there is another variant of FSM called **Mealy machines**, whose outputs depend on input symbols as well.

Digital design: Moore and Mealy machines

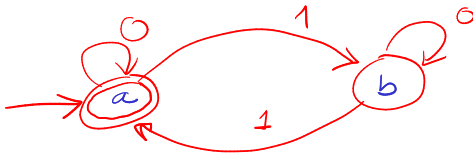
In the digital design class, you will encounter finite-state machines as well. The version we consider in this class is referred to as a **Moore machine**.

In practice, there is another variant of FSM called **Mealy machines**, whose outputs depend on input symbols as well.

Formally, they differ in output function.

- ▶ Moore machine: $G : Q \longrightarrow [0, 1]$
- ▶ Mealy machine: $G : Q \times \Sigma \longrightarrow [0, 1]$

Example: even number of 1's



- $\Sigma = \{0, 1\}$

- $Q = \{a, b\}$

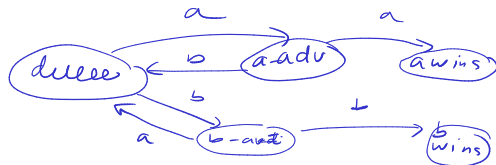
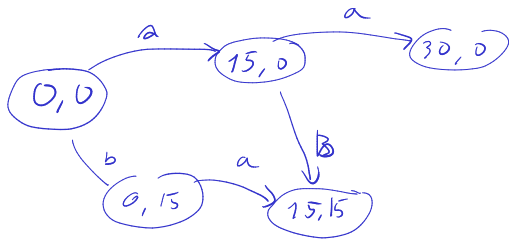
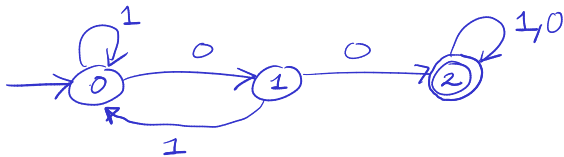
- δ :

	0	1
a	a	b
b	b	a

- $q_0 = a$

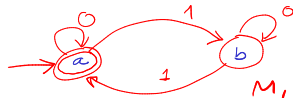
- $A = \{a\}$

Example: strings containing 00 as a substring



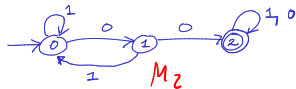
Combining DFAs

$$L_1 = L(M_1)$$



What if we want to build a DFA that accepts strings with an even number of 1's and containing 00 as a substring?

$$L(M) = L_1 \cap L_2$$



$$L_2 = L(M_2)$$



$$M_1 = (\Sigma, Q_1, \delta_1, \underline{a}, A_1)$$

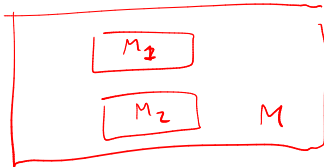
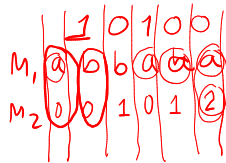
$$M_2 = (\Sigma, Q_2, \delta_2, \underline{0}, A_2)$$

$\parallel$
 $\{0, 1, 2\}$

$$(a, 0) \xrightarrow{1} (b, 0)$$

$$\downarrow 0$$

$$(b, 1)$$



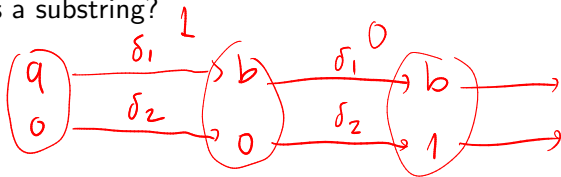
$$A_1 \times A_2$$

$$Q_1 \times Q_2 = \{(a, 0), (b, 0), (b, 1), (a, 1), (a, 2), (b, 2)\}$$

$$(a, 0)$$

Combining DFAs

What if we want to build a DFA that accepts strings with an even number of 1's and containing 00 as a substring?

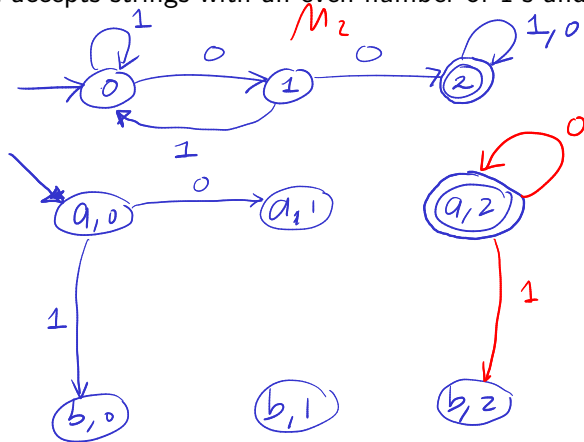
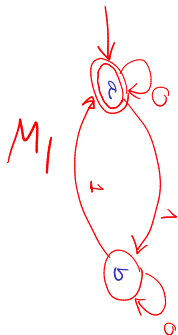


$$\begin{array}{c} (q, q') \\ \downarrow x \quad \searrow \\ (\delta_1(q, x), \delta_2(q', x)) \end{array}$$

Combining DFAs

What if we want to build a DFA that accepts strings with an even number of 1's and containing 00 as a substring?

$(a, 2)$
 $\downarrow 0 \quad \downarrow 0$
 $(a, 2)$



Product construction

Let DFA $M_1 = (\Sigma, Q_1, \delta_1, q_1, A_1)$

$M_2 = (\Sigma, Q_2, \delta_2, q_2, A_2)$

Construct DFA $M = (\Sigma, Q, \delta, q, A)$

where $L(M) = L(M_1) \cap L(M_2)$.

~~Σ~~

$Q = Q_1 \times Q_2$ - all state pairs M_1 & M_2 possible

$q = (q_1, q_2)$ - start pair M_1 & M_2

$A = A_1 \times A_2$ - only acceptive states where q_i

" $\{(a, b) \mid a \in A_1, b \in A_2\}$

$\frac{q_1 \in Q_1, A_1 \subseteq Q_1}{q_2 \in Q_2, A_2 \subseteq Q_2}$

Product construction (formally)

Given a DFA $M_1 = (\Sigma, Q_1, \delta_1, s_1, A_1)$ that accepts strings from language L_1 and $M_2 = (\Sigma, Q_2, \delta_2, s_2, A_2)$ that accepts strings from language L_2 , we can construct a DFA $M = (\Sigma, Q, \delta, s, A)$ that accepts strings from $L_1 \cap L_2$ as follows:

- ▶ Let $Q = Q_1 \times Q_2$,

Product construction (formally)

Given a DFA $M_1 = (\Sigma, Q_1, \delta_1, s_1, A_1)$ that accepts strings from language L_1 and $M_2 = (\Sigma, Q_2, \delta_2, s_2, A_2)$ that accepts strings from language L_2 , we can construct a DFA $M = (\Sigma, Q, \delta, s, A)$ that accepts strings from $L_1 \cap L_2$ as follows:

- ▶ Let $Q = Q_1 \times Q_2$,
- ▶ Let $\delta : (Q_1 \times Q_2) \times \Sigma \longrightarrow (Q_1 \times Q_2)$ be such that

$$\delta((q, q'), x) = (\delta_1(q, x), \delta_2(q', x))$$

Product construction (formally)

Given a DFA $M_1 = (\Sigma, Q_1, \delta_1, s_1, A_1)$ that accepts strings from language L_1 and $M_2 = (\Sigma, Q_2, \delta_2, s_2, A_2)$ that accepts strings from language L_2 , we can construct a DFA $M = (\Sigma, Q, \delta, s, A)$ that accepts strings from $L_1 \cap L_2$ as follows:

- ▶ Let $Q = Q_1 \times Q_2$,
- ▶ Let $\delta : (Q_1 \times Q_2) \times \Sigma \longrightarrow (Q_1 \times Q_2)$ be such that

$$\delta((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a)),$$

Product construction (formally)

Given a DFA $M_1 = (\Sigma, Q_1, \delta_1, s_1, A_1)$ that accepts strings from language L_1 and $M_2 = (\Sigma, Q_2, \delta_2, s_2, A_2)$ that accepts strings from language L_2 , we can construct a DFA $M = (\Sigma, Q, \delta, s, A)$ that accepts strings from $L_1 \cap L_2$ as follows:

- ▶ Let $Q = Q_1 \times Q_2$,
- ▶ Let $\delta : (Q_1 \times Q_2) \times \Sigma \longrightarrow (Q_1 \times Q_2)$ be such that

$$\delta((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a)),$$

- ▶ Let $s = (s_1, s_2)$, and

Product construction (formally)

Given a DFA $M_1 = (\Sigma, Q_1, \delta_1, s_1, A_1)$ that accepts strings from language L_1 and $M_2 = (\Sigma, Q_2, \delta_2, s_2, A_2)$ that accepts strings from language L_2 , we can construct a DFA $M = (\Sigma, Q, \delta, s, A)$ that accepts strings from $L_1 \cap L_2$ as follows:

- ▶ Let $Q = Q_1 \times Q_2$,
- ▶ Let $\delta : (Q_1 \times Q_2) \times \Sigma \longrightarrow (Q_1 \times Q_2)$ be such that

$$\delta((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a)),$$

- ▶ Let $s = (s_1, s_2)$, and
- ▶ Let $A = A_1 \times A_2$.

Product construction (formally)

Given a DFA $M_1 = (\Sigma, Q_1, \delta_1, s_1, A_1)$ that accepts strings from language L_1 and $M_2 = (\Sigma, Q_2, \delta_2, s_2, A_2)$ that accepts strings from language L_2 , we can construct a DFA $M = (\Sigma, Q, \delta, s, A)$ that accepts strings from $L_1 \cup L_2$ as follows:

- ✓ ▶ Let $Q = Q_1 \times Q_2$,
- ✓ ▶ Let $\delta : (Q_1 \times Q_2) \times \Sigma \longrightarrow (Q_1 \times Q_2)$ be such that

$$\delta((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a)),$$

- ✓ ▶ Let $s = (s_1, s_2)$, and
- ▶ Let $A = A_1 \times A_2$.

$$A = \{ (q_1, q_2) \in Q_1 \times Q_2 \mid q_1 \in A_1, \text{ and } q_2 \in A_2 \}$$

Product construction (formally)

Given a DFA $M_1 = (\Sigma, Q_1, \delta_1, s_1, A_1)$ that accepts strings from language L_1 and $M_2 = (\Sigma, Q_2, \delta_2, s_2, A_2)$ that accepts strings from language L_2 , we can construct a DFA $M = (\Sigma, Q, \delta, s, A)$ that accepts strings from $L_1 \cap L_2$ as follows:

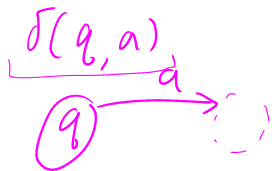
- ▶ Let $Q = Q_1 \times Q_2$,
- ▶ Let $\delta : (Q_1 \times Q_2) \times \Sigma \longrightarrow (Q_1 \times Q_2)$ be such that

$$\delta((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a)),$$

- ▶ Let $s = (s_1, s_2)$, and
- ▶ Let ~~$A = A_1 \times A_2$~~ .

$$A = \{ (q_1, q_2) \in Q_1 \times Q_2 \mid \underline{q_1 \in A_1}, q_2 \notin A_2 \}$$

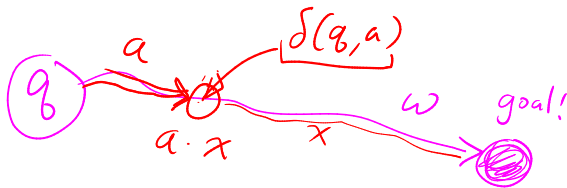
Recall the definition of $\delta^*(q, w)$, i.e., $\delta^*: Q \times \Sigma^* \rightarrow Q$.



$$\delta: Q \times \Sigma \rightarrow Q$$

Recall the definition of $\delta^*(q, w)$, i.e.,

$$\delta^*(q, w) = \begin{cases} q & \text{if } w = \varepsilon, \\ \underbrace{\delta^*(\delta(q, a), x)} & \text{if } w = ax \text{ where } a \in \Sigma \end{cases}$$



Recall the definition of $\delta^*(q, w)$, i.e.,

$$\delta^*(q, w) = \begin{cases} q & \text{if } w = \varepsilon, \\ \delta^*(\delta(q, a), x) & \text{if } w = ax \text{ where } a \in \Sigma \end{cases}$$

Lemma 1

$\delta^*((q_1, q_2), w) = (\delta_1^*(q_1, w), \delta_2^*(q_2, w))$ for any string w .

Proof.

We prove by induction. **I.H.:** Assume that $\delta^*((q_1, q_2), x) = (\delta_1^*(q_1, x), \delta_2^*(q_2, x))$, for any string x such that $|x| < |w|$.

Case $w = \varepsilon$: $\delta^*((q_1, q_2), \varepsilon) = (q_1, q_2) = (\delta_1^*(q_1, \varepsilon), \delta_2^*(q_2, \varepsilon))$

Case $w = ax$ where $a \in \Sigma, x \in \Sigma^*$

$$\delta^*((q_1, q_2), ax) = \delta^*(\delta((q_1, q_2), a), x) = \delta^*((\delta_1(q_1, a), \delta_2(q_2, a)), x)$$



Recall the definition of $\delta^*(q, w)$, i.e.,

$$\delta^*(q, w) = \begin{cases} q & \text{if } w = \varepsilon, \\ \delta^*(\delta(q, a), x) & \text{if } w = ax \text{ where } a \in \Sigma \end{cases}$$

Lemma 1

$\delta^*((q_1, q_2), w) = (\delta_1^*(q_1, w), \delta_2^*(q_2, w))$ for any string w .

Proof.

for any q_1, q_2

We prove by induction. I.H.: Assume that $\delta^*((\underline{q_1}, \underline{q_2}), \underline{x}) = (\underline{\delta_1^*(q_1, x)}, \underline{\delta_2^*(q_2, x)})$, for any string x such that $|x| < |w|$.

$$\begin{aligned} \text{mb} \quad &= \delta^*((\delta_1(q_1, a), \delta_2(q_2, a)), x) = (\underbrace{\delta_1^*(\delta_1(q_1, a), x)}_{\substack{\text{by I.H.} \\ q_1^*}}, \delta_2^*(\delta_2(q_2, a), x)) \\ &= (\delta_1^*(q_1, ax), \delta_2^*(q_2, ax)) \\ &= (\delta_1^*(q_1, w), \delta_2^*(q_2, w)) \quad \square \end{aligned}$$

Correctness

From the previous lemma, we have that

$$\delta^*(s, w) = \delta^*((s_1, s_2), w)$$

Correctness

From the previous lemma, we have that

$$\begin{aligned}\delta^*(s, w) &= \delta^*((s_1, s_2), w) \\ &= (\delta_1^*(s_1, w), \delta_2^*(s_2, w))\end{aligned}$$

of Lemma 1

Correctness

From the previous lemma, we have that

$$\begin{aligned}\delta^*(s, w) &= \delta^*((s_1, s_2), w) \\ &= (\delta_1^*(s_1, w), \delta_2^*(s_2, w))\end{aligned}$$

Thus, for an input w , M would reach the state $(\delta_1^*(s_1, w), \delta_2^*(s_2, w))$; it accepts w when

$$(\delta_1^*(s_1, w), \delta_2^*(s_2, w)) \in A_1 \times A_2.$$

This implies that M accepts w when $\delta_1^*(s_1, w) \in A_1$ and $\delta_2^*(s_2, w) \in A_2$, i.e., M accepts w iff M_1 and M_2 accept w .

Finally, we conclude that M accepts strings from language $L_1 \cap L_2$.

Language of a DFA

$L(M)$

For a DFA M , let $L(M)$ be the set of all strings that M accepts. More formally, for $M = (\Sigma, Q, \delta, s, A)$,

$$L(M) = \{w \in \Sigma^* \mid \delta^*(s, w) \in A\}.$$

We refer to $L(M)$ as the language of M .

Closure

Lemma 2

If L_1 and L_2 are languages of some DFAs M_1 and M_2 , we have that

- ▶ *there is a DFA M that accepts $L_1 \cap L_2$,*

Lemma 2

If L_1 and L_2 are languages of some DFAs M_1 and M_2 , we have that

- ▶ *there is a DFA M that accepts $L_1 \cap L_2$,*
- ▶ *there is a DFA M that accepts $L_1 \cup L_2$,*

Closure

Lemma 2

If L_1 and L_2 are languages of some DFAs M_1 and M_2 , we have that

- ▶ *there is a DFA M that accepts $L_1 \cap L_2$,*
- ▶ *there is a DFA M that accepts $L_1 \cup L_2$,*
- ▶ *there is a DFA M that accepts $L_1 \setminus L_2$,*

Closure

Lemma 2

If L_1 and L_2 are languages of some DFAs M_1 and M_2 , we have that

- ▶ *there is a DFA M that accepts $L_1 \cap L_2$,*
- ▶ *there is a DFA M that accepts $L_1 \cup L_2$,*
- ▶ *there is a DFA M that accepts $L_1 \setminus L_2$,*
- ▶ *there is a DFA M that accepts $\Sigma^* \setminus L_1$,*

Automatic languages²

Definition (for now)

A language L is “**automatic**” if there is a DFA M such that $L(M) = L$.

²Taken directly from Erickson's lecture notes

Automatic languages²

Definition (for now)

A language L is “**automatic**” if there is a DFA M such that $L(M) = L$.

Lemma 3

If L_1 and L_2 are automatic languages over alphabet Σ , then

► $L_1 \cap L_2$,

²Taken directly from Erickson's lecture notes

Automatic languages²

Definition (for now)

A language L is “**automatic**” if there is a DFA M such that $L(M) = L$.

Lemma 3

If L_1 and L_2 are automatic languages over alphabet Σ , then

- ▶ $L_1 \cap L_2$,
- ▶ $L_1 \cup L_2$,

²Taken directly from Erickson's lecture notes

Automatic languages²

Definition (for now)

A language L is “**automatic**” if there is a DFA M such that $L(M) = L$.

Lemma 3

If L_1 and L_2 are automatic languages over alphabet Σ , then

- ▶ $L_1 \cap L_2$,
- ▶ $L_1 \cup L_2$,
- ▶ $L_1 \setminus L_2$, *and*

²Taken directly from Erickson's lecture notes

Automatic languages²

Definition (for now)

A language L is “**automatic**” if there is a DFA M such that $L(M) = L$.

Lemma 3

If L_1 and L_2 are automatic languages over alphabet Σ , then

- ▶ $L_1 \cap L_2$,
- ▶ $L_1 \cup L_2$,
- ▶ $L_1 \setminus L_2$, and
- ▶ $\Sigma^* \setminus L_1$,

are also automatic.

²Taken directly from Erickson's lecture notes

Automatic languages²

Definition (for now)

A language L is “**automatic**” if there is a DFA M such that $L(M) = L$.

Lemma 3

If L_1 and L_2 are automatic languages over alphabet Σ , then

- ▶ $L_1 \cap L_2$,
- ▶ $L_1 \cup L_2$,
- ▶ $L_1 \setminus L_2$, and
- ▶ $\Sigma^* \setminus L_1$,

are also automatic.

The set of automatic languages is closed under these boolean operations.

²Taken directly from Erickson's lecture notes